

Quartus Prime Lite

instalación y configuración

Para la simulación de los diseños hechos en Quartus hay dos opciones: la primera es instalar ModelSim Starter edition que no requiere licencia, la segunda es instalar Questa que requiere una licencia paga o estudiantil. En este instructivo vamos con la primera opción.

1. Descargar los instaladores

- **Quartus:** <https://www.intel.la/content/www/xl/es/software-kit/849770/>
- **ModelSim:** <https://www.intel.com/content/www/us/en/software-kit/750666/>

Seleccionar arriba la última versión y el sistema operativo en el que lo vamos a instalar (Windows o Linux).

2. Ejecutar Quartus

Windows

Cliqueamos el ejecutable qinst*.exe

Linux

```
cd ~/Downloads
sudo chmod 744 qinst*
./qinst*
```

NOTA: Para Linux no es necesario instalarlo ni ejecutarlo con privilegios de administrador.

3. Seleccionar los componentes que vamos a instalar y la ruta de instalación

Como optamos por ModelSim, solo necesitamos instalar Quartus y las bibliotecas para la Cyclone IV (porque es la FPGA que vamos a necesitar).

▼	☒ Quartus® Prime Lite Edition (Free)			
	☒ Quartus® Prime (includes Nios II EDS)	2.09 GB	8.58 GB	0%
▼	☐ Questa*-Altera FPGA and Starter Editions	1.64 GB	5.70 GB	
	☐ Starter Edition			
▼	☒ Devices			
	☐ Arria® II device support	0.49 GB	0.52 GB	
	☒ Cyclone® IV device support	0.46 GB	0.50 GB	0%
	☐ Cyclone® 10 LP device support	0.26 GB	0.29 GB	
	☐ Cyclone® V device support	1.34 GB	1.40 GB	
	☐ MAX® II, MAX® V device support	0.01 GB	0.01 GB	
	☐ MAX® 10 FPGA device support	0.28 GB	0.36 GB	
▼	☐ Add-ons and Standalone Software			
	☐ Quartus® Prime Programmer and Tools 24.1std.0.1077	0.41 GB	1.61 GB	
	☐ Quartus® Prime Help 24.1std.0.1077	0.27 GB	0.49 GB	
	☐ Ashling* RiscFree* IDE for Altera 24.1std.0.1077	0.84 GB	3.18 GB	

En cuanto a la ruta de instalación, conviene dejar la ruta que viene por defecto si no hay motivos para cambiarla. De lo contrario, hay que tener en cuenta los eventuales problemas de permisos de acceso, que sobre todo aplica para Linux.

4. Descargar e instalar Quartus

5. Ejecutar e instalar ModelSim (Starter edition)

Windows

Cliqueamos el ejecutable ModelSim*.exe

Linux

```
cd ~/Downloads
sudo chmod 744 ModelSim*
./ModelSim*
```

NOTA: mismas observaciones que Quartus sobre los privilegios de administrador para Linux, mismas observaciones sobre la carpeta de instalación.

6. Instalar bibliotecas restantes (SOLO LINUX)

ModelSim requiere bibliotecas de 32 bits, por lo tanto debemos activar el repositorio de 32 bits. Con apt hacemos:

```
sudo dpkg --add-architecture i386
sudo apt install libxext6:i386 libxft2:i386
```

NOTA: dependiendo de la distribución puede que sea necesario instalar otras versiones u otras bibliotecas adicionales. Para esto, conviene intentar ejecutar ModelSim

```
~/intelFPGA/20.1/modelsim_ase/linuxaloem/vsim
```

e ir leyendo los errores de dependencias requeridas hasta que hayamos instalado todas, cuando esto ocurra nos dejará ejecutarlo. Si este paso no está bien resuelto, cuando queramos correr la primera simulación en el waveform de Quartus el simulador se va a colgar cuando esté generando la salida.

7. Ejecutar Quartus

8. Vincular ModelSim a Quartus

Tools → Options → EDA Tool Options → ModelSim , nos aseguramos que aparezca la ruta directa a la carpeta del ejecutable:

- Windows: C:/intelFPGA/20.1/modelsim_ase/win32aloem
- Linux: /home/<usuario>/intelFPGA/20.1/modelsim_ase/linuxaloem

Crear un proyecto

File → New → Quartus Prime Project , elegimos carpeta y nombre para el proyecto. Elegimos un proyecto vacío, no necesitamos añadir ningún archivo, seleccionamos el modelo de la FPGA para el que vayamos a sintetizar el diseño (*¿cuál es?*), no necesitamos configurar nada en EDA Tools Settings, finalizamos.

Vista jerárquica vs vista de archivos

Por defecto nos va a mostrar la vista jerárquica, que no tiene mucha utilidad para diseñar. Si vamos a la barra de la izquierda, arriba podemos cambiar para que nos muestre la vista de archivos (es decir, navegar por los archivos que son parte del proyecto).

Crear el punto de entrada

Vamos a crear un archivo de tipo esquemático, que será el punto de entrada para la compilación. File → New → Design Files → Block diagram/Schematic file . Por defecto, Quartus piensa que el punto de entrada es un archivo que tiene el mismo nombre que el proyecto. A mí me gusta llamarlo "top", entonces lo guardo como y le pongo de nombre "top(.bdf)". Luego voy a Assignments → Settings → General y le digo que mi punto de entrada se llama "top" (sin extensión).

Gestión de archivos

El proyecto se crea en un archivo *.qpf que tiene toda la información con los archivos que forman parte de él. Por este motivo, lo más práctico es crear los archivos dentro del propio editor de Quartus. Si lo creamos desde fuera, necesitamos avisarle a nuestro proyecto para que reconozca los nuevos archivos creados como parte del proyecto.

En este sentido, conviene siempre abrir el proyecto entero (File → open project) en lugar de abrir un archivo aislado (File → open).

En el caso de que queramos agregar un archivo que no fue creado en Quartus, vamos a Project → Add/remove files.

Generar un nuevo bloque

Hay dos opciones para generar bloques: a partir de un archivo de (System-)Verilog o a partir de un archivo esquemático. Generalmente lo más práctico es la primera opción mencionada, aunque vale la pena tener ambas opciones en consideración.

Supongamos que queremos hacerlo a partir de un .sv. File → New → SystemVerilog HDL File. Escribimos el código para nuestro módulo (delimitado con module/endmodule) y cuando terminemos lo guardamos y vamos a File → Create/update → Create symbol files from current file. Ahora vamos a poder usar el módulo que hayamos definido como un bloque esquemático (caja negra) que lo podemos usar con clic derecho → Insert → Symbol.

Generar un diagrama de señales (simulación)

Los waveforms son archivos que nos permiten testear todo el circuito que armamos en el archivo principal del proyecto. Para eso primero tenemos que pasar la *compilación y síntesis*. Luego vamos a File → New → Verification → University program (VWF). Hacemos clic en el archivo creado, vamos a las opciones de simulación y quitamos la opción de –novopt en ambas pestañas. Generamos el Testbench y ya podemos generar la salida.

Insertar una memoria

Ip Catalog → Library → On Chip Memory → RAM: 2-PORT. Configuramos la RAM y dejamos vacío en la parte para definir el archivo de inicialización. Le pedimos que genere el archivo de verilog y el bloque esquemático.

NOTA: El estándar de RISC V usa un ancho de 8 bits por cada dirección.

Generar el código de inicialización para la memoria

Vaya a <https://riscvasm.lucasteske.dev/>. Escriba su programa en assembler, siguiendo los estándares del set de instrucciones de RISC V.

NOTA: tenga en cuenta que por defecto el compilador genera un offset de 0x10000 entre la zona del programa (ROM) y la zona de los datos (RAM).

Cuando termine, haga clic en "build" y guarde la salida del hex dump en un archivo. Todavía no está listo porque Quartus exige información adicional sobre la memoria, para lo cual lo convertiremos a un archivo .mif que será el archivo que le pasaremos al bloque generado de la RAM. Se adjunta el script hex2mif.py que convierte el formato del archivo. Se ejecuta pasando el nombre del archivo terminado en ".hex" como primer parámetro.

Se recomienda ser ordenado, una estrategia es crear una carpeta dentro de la carpeta del proyecto que se llame "scripts" (esta carpeta no es parte del proyecto pues se crea desde afuera). Dentro de esta carpeta guardamos los archivos con los programas en assembler, los .hex (obtenidos del hex dump) y los .mif generados con el script de Python.

Una vez tengamos en .mif, hacemos clic sobre el bloque de la RAM para que abra el asistente y vamos hasta donde pide el archivo. Indicamos la ruta, guardamos y regeneramos el bloque. Ahora en la simulación debería aparecer la memoria inicializada con el programa.

Links útiles

https://cmput229.github.io/229-labs-RISCV/RISC-V-Examples_Public/example.html: ejemplos básicos hechos en assembler para RISC V

<https://marz.utk.edu/my-courses/cosc230/book/example-risc-v-assembly-programs/>: ejemplos de códigos más avanzados hechos en assembler para RISC V

<https://projectf.io/posts/riscv-cheat-sheet/>: cheat sheet sobre el estándar del set de instrucciones. Como observación, conviene respetar la notación descriptiva del registro, si bien en RISC V todos los registros son iguales, el estándar nos recomienda que respetemos convenciones que le asignan funciones específicas a ciertos registros.